RESCUE OPERATIONS SYSTEM (ARDMS)

Advanced Rescue & Disaster Management Suite

SYSTEM TECHNICAL STACK

Technical Architecture and Implementation Guide

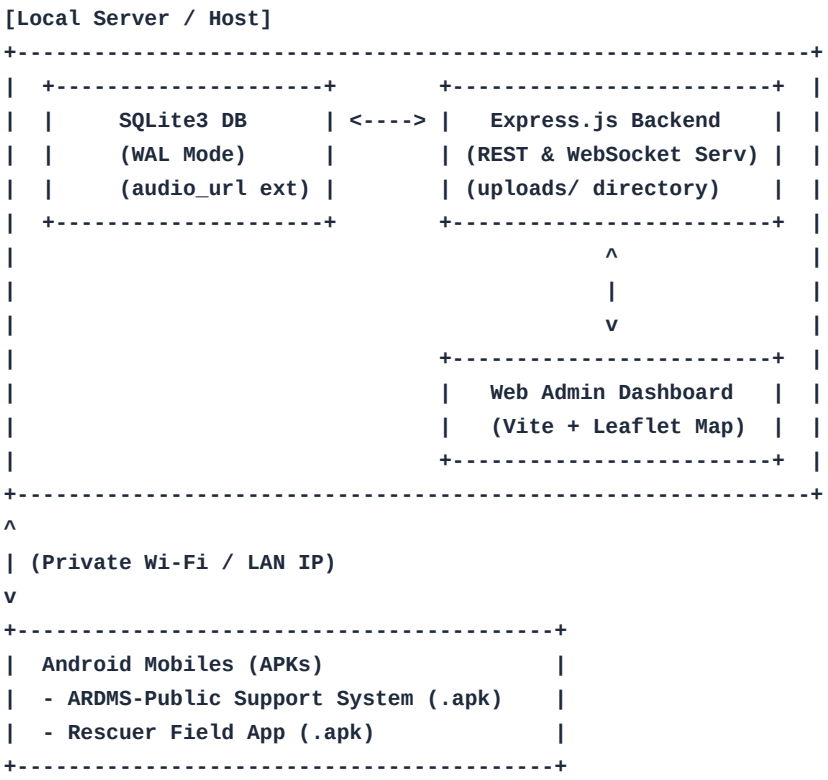
Rescue Operations System (ARDMS) - Complete System Manual

Welcome to the official system manual, deployment guide, and troubleshooting handbook for the **Rescue Operations System (ARDMS)**. This document acts as a comprehensive technical guide for developers, students, and administrators to deploy, configure, and troubleshoot the entire emergency management network.

---

1. Technical Stack & Architecture

The Rescue Operations System is divided into four highly-optimized, interconnected layers designed for low-network resilience, real-time telemetry, and emergency response operations:



1.1. Backend Command Core (`system-backend`)

- **Runtime:** Node.js

- **Framework:** Express.js (v5)

- **Database:** SQLite3 with **WAL (Write-Ahead Logging) Mode** enabled for concurrent transaction support and database lock prevention during emergency peaks.

- *Dynamic Migrations:* On boot, the server checks the database schema and automatically adds the

`audio_url` column if it is missing, preserving existing tables without data loss.
- **Real-time Server:** WebSockets (`ws`) for live rescuer GPS telemetry.
- **Authentication:** Stateless JSON Web Tokens (`jsonwebtoken`) and `bcryptjs` password hashing.
- **Storage Ingestion:** REST endpoint decodes incoming base64 SOS audio files and records them in physical server directories (`/uploads/sos_audio_[timestamp].wav`) linked dynamically via SQLite fields.

1.2. Web Admin Command Center (`web-admin`)

- **Framework:** React-based Single Page Application (Vite compiler).
- **Map & Route Engine:** Leaflet.js mapping library.
- *Dynamic Map Clustering:* Static markers removed completely. The map clusters and displays active field incidents dynamically using real-time operational data.
- *Tactical Route Auto-Erase:* Actively monitors tracking states and instantly wipes the Leaflet polyline route from the screen once a rescuer completes, resolves, or declines a task.
- *Tactical Grouping & Task Cascading:* Admins can bundle multiple SOS requests into "Tactical Clusters" for squad dispatch. The backend automatically deduplicates individual assignments to prevent mobile notification spam. Completing a Tactical Cluster from either the mobile app or Web Admin safely cascades the `completed` status down to all child SOS requests.
- **Media Columns Split:** Separated active grids and history tables to offer distinct columns for `IMAGE` (embedded thumbnails + lightboxes) and `AUDIO` (inline `<audio controls>` player + download link).

1.3. ARDMS-Public Support System (`public-sos-app`)

- **Framework:** React Native (bare Expo SDK ~54) compiled natively inside `android/` subprojects.
- **Location Engine:** `expo-location` for continuous GPS tracking.
- **Stability Fix:** Pre-try block scoped `payload` definition eliminates crashes during offline queue catches.
- **Buffer Timer Display:** Displays `#selectionBufferCooldownBanner` showing real-time countdown blocks directly on the selection home page if a user is in a cool-down lockout state.
- **Cache & Sync Engine:** Implements local `AsyncStorage` caching. It saves reporting history locally and queues offline reports, synchronizing them to the host server automatically when a network connection is re-established.
- **Session Preservation:** Custom clear-cache settings preserve active `sosUser` tokens (`citizen_user` / `citizen_token`) so that standard data scrub operations do not force logouts.

1.4. Rescuer Field App (`rescuer-app`)

- **Framework:** React Native (bare Expo SDK ~54) compiled natively.
- **Maps Integration:** `react-native-maps` for tracking other rescuers and incident locations.
- **All-Status History Integration:** Queries the Express backend to dynamically load historical tasks (including completed, declined, and ongoing status states) in the history page.
- **Offline Caching:** Full `localStorage` mirroring displays the rescue logs even if the app is launched in completely dead network zones.
- **Session Preservation:** Modified clean routines safeguard `rescue_user_v3` details to avoid lockouts under harsh field environments.

1.5. Autonomous Task Management (AI & Buffer System)

- **AI Auto-Assignment Module:** An automated background process powered by **Google Gemini 1.5 Pro AI**. It intelligently processes live situational data, interpreting JSON telemetry payloads to assign new incoming SOS requests to the most appropriate rescuer. **The AI enforces a 50-meter GPS radius proximity constraint for assignments. It evaluates the exact coordinates of active rescuers to route the task to the nearest, most available rescuer.**
- **Reassignment Buffer System:** Implements a dynamic fail-safe timeout logic. If an assigned rescuer goes offline, ignores the task beyond the configured buffer interval, or explicitly declines the mission, the system instantly revokes the assignment and autonomously triggers the AI to re-evaluate. **Crucially, the backend automatically clears any previous 'declined' or 'ignored' history for that specific task to ensure the AI does not falsely skip valid rescuers during reassignment loops.**
- **Rescuer Task Siren System:** Field units receive an intrusive audio alert (siren) whenever a critical task is assigned. **To prevent phantom sirens, all dispatch commands are sent via strictly targeted WebSocket `send(deviceId)` calls, ensuring only the exact authorized rescuer receives the assignment trigger.**
- **WebView Cache Evasion:** Prevents React Native WebView from aggressively caching mission data by implementing timestamp-based cache-busters on fetch endpoints (e.g., `?_t=${Date.now()}`), guaranteeing field units always see the exact, real-time status of reassigned or completed tasks.
- **Web Admin UX Integrity:** Enforces visual CSS rules (like `flex-wrap` and scrollable overflow containers) across the dashboard to ensure dense tactical data and action buttons don't break the layout on smaller command screens.

---

2. Deployment Guide

Deploying the system locally involves setting up a private communication subnet to bridge your Windows PC server (localhost) and mobile devices. Below are two deployment methodologies.

2.1. Private Wi-Fi LAN Setup

This is the standard approach using a local Wi-Fi router or mobile hotspot.

1. Connect to the Same Network: Ensure your Windows computer and the Android devices are connected to the exact same Wi-Fi router or phone hotspot.

2. Find Your Computer's LAN IP:

- Open **PowerShell** (Press Windows Key, type `powershell`, and press Enter).
- Type `ipconfig` and press Enter.
- Look for the section marked **`Wireless LAN adapter Wi-Fi`** and read the **`IPv4 Address`** (e.g., `192.168.1.4`).

3. Configure Windows Firewall: Open PowerShell as **Administrator** (Right-click PowerShell -> Run as Administrator) and run:

```
New-NetFirewallRule -DisplayName "Rescue Backend Port 3001" -Direction Inbound -LocalPort 3001 -Protocol TCP -Ac
```

4. Synchronize Endpoints: Open CMD, navigate to the workspace, and run:

```
python sync_apps.py
```

5. Start Server: Navigate to `system-backend` and run `npm start`.

6. Install APKs & Connect via Manual IP Feeding: Copy files from `Output_APKs/` to your Android devices and install them. **Note: Because the mobile apps support manual IP configuration, rebuilding the APKs is NOT required when changing Wi-Fi networks. Simply launch the app, type the PC's new local LAN IP (e.g., `192.168.1.4`) into the connection interface, and it will connect seamlessly.**

2.2. Private Cellular Connection Setup

If a Wi-Fi router is unavailable, range requirements extend across miles, or standard commercial infrastructure has collapsed, you can establish a private cellular-to-server connection using four distinct techniques.

2.2.1. Private Network-in-a-Box (NIB) Cellular Base Station Setup

This advanced operational setup deploys a portable, fully self-contained cellular base station (eNodeB/gNodeB Software Defined Radio + Mobile Core EPC/5GC) containing an internal private Windows computer acting as the server. Rescuers connect using mobile devices equipped with private NIB-SIM cards.

A. Pre-Installation Setup Requirements (Host Computer Preparation)

Before initializing the deployment, you must load the following software environments, hardware drivers, and utilities onto the NIB Windows server host computer:

- **SDR Transceiver Hardware Drivers:**

- **USRP Hardware Driver (UHD):** Required to interface Ettus USRP B210 transceivers. Install the official National Instruments UHD package.

- **BladeRF Windows Driver & Firmware:** For Nuand bladeRF units. Install the latest bladeRF driver and matching FPGA firmware.

- **Zadig USB Driver Tool:** A utility to replace default Windows USB drivers with **`WinUSB`** for your specific SDR device. Connect the SDR, run Zadig, click *Options -> List All Devices*, select the SDR transceiver, and click *Replace Driver* to swap the manufacturer driver with the generic USB driver so user-space programs can directly access it.

- **WSL2 (Windows Subsystem for Linux) Setup:**

- Since leading open-source cellular cores (e.g. Open5GS, srsRAN) compile and run natively on Linux, the Windows host requires **WSL2 with Ubuntu 22.04 LTS** installed.

- Execute inside PowerShell:

```
wsl --install -d Ubuntu-22.04
wsl --update
```

- **USBIPD-WIN:** Command-line tool to pass-through physical USB SDR hardware from the Windows host environment into the virtualized WSL2 Linux guest. Install on Windows and run:

```
usbipd wsl list
usbipd wsl attach --busid <SDR_BUS_ID>
```

- **USIM Programming Kit:**

- **PC/SC Smartcard Reader Drivers:** Windows drivers for chip-card writers (e.g., ACR38/ACR39).

- **SIM Card Writer Software (PySIM):** Python-based utility (`pysim-prog`) or generic GUI card writers to burn PLMN, IMSI, and authentication keys (Ki/OPc) into blank programmable USIM cards.

- **System Backend & Database Tools:**
- **Node.js (v18+) & Git:** Required to fetch, compile, and run the backend Express services.
- **DB Browser for SQLite:** Visual database manager to inspect the SQLite `rescue.db` schema and transaction queues.

B. Step-by-Step NIB Setup & Deployment

- **Step 1: SIM Card Provisioning:**
- To pair the mobile devices, obtain blank programmable USIM cards and a USB Smart Card Writer.
- Program each SIM card with the private NIB network identifiers matching the EPC/5GC Subscriber Database:
- **PLMN ID (MCC/MNC):** e.g., `001/01` (Test network) or `999/99` (Private network).
- **IMSI:** e.g., `001010000000001` (Incremented sequentially for each rescue team handset).
- **Authentication Keys (K, OPc):** Retrieve these secret hexadecimal keys from the NIB server core configurations and burn them into the SIM card.
- Insert the programmed SIM card into the Android smartphone.
- **Step 2: Mobile APN Configuration:**
- On the Android device, go to **Settings** -> **Network & Internet** -> **Mobile Network** -> **Access Point Names (APNs)**.
- Tap the **`+`** icon to add a new custom APN:
- **Name:** `Rescue NIB Core`
- **APN:** `rescue.nib`
- **APN Type:** `default,supl`
- **APN Protocol:** `IPv4/IPv6`
- **APN Roaming Protocol:** `IPv4`
- Tap the three dots and select **Save**. Select the new `Rescue NIB Core` APN.
- Turn **Cellular Data** and **Data Roaming** to **ON**. The phone will scan, authenticate via the private SIM, and connect to the private base station signal, displaying the LTE/5G network indicator.
- **Step 3: NIB Server Local Core Routing:**
- The NIB's internal Windows computer runs the cellular core. The EPC/5GC virtual network interface assigns a gateway IP to the mobile core client subnet (typically `10.45.0.1` as the PGW/UPF gateway interface).
- Run **PowerShell** on the NIB Windows server to verify the active core interfaces:

```
ipconfig
```

- Note the IP address of the local network interface named **`Open5GS-TUN`** or **`srsran-tun`** (usually `10.45.0.1`). This is the static IP address that all connected smartphones will use to communicate with the ARDMS backend.
- **Step 4: Windows Firewall Administrator Rules:**
- On the NIB Windows PC, open **PowerShell as Administrator** and add the inbound TCP traffic rule for port `3001` on the virtual TUN interface:

```
New-NetFirewallRule -DisplayName "Rescue NIB Express Port" -Direction Inbound -LocalPort 3001 -Protocol TCP -Act
```

- **Step 5: Synchronize and Start the Rescue Platform:**
- Open CMD, navigate to the workspace directory:

```
cd "C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026"
```

- Run the synchronizer script and pass your specific NIB core TUN IP as an argument (e.g., `10.45.0.1`) to override dynamic LAN detection and configure the API routes to bind to the virtual cellular base station gateway:

```
python sync_apps.py 10.45.0.1
```

- Start the backend system:

```
cd system-backend
npm start
```

- **Step 6: Step 7: Verification & Sideloading (Detailed Testing Protocol):**

- **Developer Options & Debugging Setup:**

- On the Android smartphone, go to **Settings** -> **About Phone**, and tap the **`Build Number`** seven (7) times sequentially until the screen displays *"You are now a developer!"*.

- Go back to **Settings** -> **System** -> **Developer Options** and toggle **`USB Debugging`** to **ON**. Connect the device to your PC using a high-quality USB data cable.

- **ADB Client Deployment:**

- On the Windows host, download and extract Android Platform Tools. Open CMD, navigate to the extracted tools folder, and verify the handset connection:

```
adb devices
```

(Ensure the screen displays your phone's serial number followed by "device".)

- Sideload the compiled application APK binaries onto the field device directly:

```
adb install "C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026\Output_APKs\rescuer-app-release.apk"
adb install "C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026\Output_APKs\public-sos-app-release.apk"
```

- **Cellular Layer Registration Verification:**

- In your NIB Server WSL2 terminal, monitor the Mobile Core EPC/5GC logs in real-time to confirm radio attachment:

- *For Open5GS MME core:* Run `tail -f /var/log/open5gs/mme.log` and verify the log shows **`Attach Request received`**, followed by cryptographic Milenage authentication success, and **`Initial Context Setup Request [IMSI: 001010000000001]`**.

- *For srsRAN EPC core:* Run `tail -f /tmp/epc.log` to watch active subscriber IMSI attach and session tunnel setups.

- **Client Network Routing Check:**

- Open the **Termux** application (terminal emulator) on the Android client smartphone and execute a manual packet ping against the NIB server's gateway IP on the private TUN network:

```
ping -c 5 10.45.0.1
```

(Verify that 5 packets are transmitted, 5 are received, and latency displays round-trip times, confirming active RF-IP datapath routes.)

- **Database Ingestion & Validation:**

- Launch the newly sideloaded **ARDMS Rescuer** or **Public App** on the device. Click the SOS distress button to send a test emergency event.

- On the Windows server computer, open the database using **DB Browser for SQLite** (located at `C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026\rescue.db`).

- Go to **Execute SQL** tab and run:

```
SELECT id, category, location, timestamp FROM rescue_requests ORDER BY timestamp DESC LIMIT 1;
```

- Verify the table successfully contains your mobile's generated category, GPS coordinates, and current timestamp. This confirms end-to-end telemetry pipeline validation!

2.2.2. Physical USB Tethering (Direct Local Cellular Bridge)

This approach leverages the mobile device's LTE/5G connection to form a direct, physical local subnet with your Windows PC server.

- **Step 1: Connect via USB Cable:** Connect your Android phone to your Windows PC using a high-quality USB data cable.
- **Step 2: Enable USB Tethering on Phone:** On the Android device, go to **Settings** -> **Network & Internet** -> **Hotspot & Tethering**, and toggle **USB Tethering** to **ON**.
- **Step 3: Discover Tethering Subnet IP:**
- Open **PowerShell on your Windows PC** (Press Windows Key, type `powershell`, and press Enter).
- Execute:

```
ipconfig
```

- Scroll through the output to find a new adapter named **`Ethernet adapter Ethernet 2`** or **`NDIS Internet Sharing Device`**.
- Note the **`IPv4 Address`** assigned to your computer on this tethering bridge (usually in the range `192.168.42.X` or `192.168.43.X`, e.g., `192.168.42.129`).
- **Step 4: Configure Inbound Firewall Rules:**
- Open **PowerShell as Administrator** (Right-click PowerShell -> Run as Administrator) and execute:

```
New-NetFirewallRule -DisplayName "Rescue Backend USB Tethering" -Direction Inbound -LocalPort 3001 -Protocol TCP
```

- **Step 5: Synchronize and Rebuild:**
- Open CMD (Press Windows Key, type `cmd`, and press Enter), navigate to the root workspace `C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026`, and run:

```
python sync_apps.py
```

- **Step 6: Start Server & Install App:**
- Navigate to the `system-backend` directory in CMD and start the server:

```
cd system-backend
npm start
```

- Side-load the compiled APKs from `Output_APKs/` onto the tethered phone, open the app, and test local real-time transmission.

2.2.3. Phone Portable Hotspot Gateway Setup

The phone acts as the cellular router, and the Windows PC joins its network.

- **Step 1: Enable Hotspot on Phone:** On your phone, go to **Settings** -> **Portable Hotspot** and turn it **ON**. Ensure your phone's Cellular Data (4G/5G) is active.
- **Step 2: Connect PC to Hotspot:** On your Windows PC, click the Wi-Fi icon in the taskbar and select your phone's Hotspot name. Enter the password to connect.
- **Step 3: Identify Gateway IP:**
- Open **PowerShell** and run `ipconfig`.
- Look under **`Wireless LAN adapter Wi-Fi`**.
- The **`IPv4 Address`** is your PC's IP, and the **`Default Gateway`** is your phone's IP.
- **Step 4: Configure Firewall Inbound Access:**
- Open **PowerShell as Administrator** and run:

```
New-NetFirewallRule -DisplayName "Rescue Backend Hotspot" -Direction Inbound -LocalPort 3001 -Protocol TCP -Acti
```

- **Step 5: Bind and Boot:**

- Run CMD, navigate to the workspace, and synchronize:

```
python sync_apps.py
```

- Start the backend server (`npm start`) and begin testing.

2.2.4. Tailscale Private Mesh VPN (Over LTE/5G Cellular Networks)

If your Windows server is in one location and your rescuers are miles away on cellular networks (without sharing a Wi-Fi or tethering connection), you can create a secure, encrypted **Virtual Private Mesh Subnet** using Tailscale to bypass Carrier-Grade NAT (CGNAT).

- **Step 1: Install Tailscale on Windows:** Download and install the Tailscale client from [Tailscale.com](https://tailscale.com/) on your Windows PC. Log in to create your private mesh network ("Tailnet").
- **Step 2: Install Tailscale on Android:** Download and install the Tailscale app from the Google Play Store on all testing Android phones. Sign in with the exact same account.
- **Step 3: Obtain Static VPN IPs:**
- Once connected, Tailscale assigns a static, secure private IP (in the `100.X.Y.Z` range, e.g., `100.82.140.23`) to your Windows PC. This IP is unique and permanent.
- Your Android devices are also assigned their own `100.X` IPs.
- **Step 4: Synchronize & Open Inbound Ports:**
- Open **PowerShell as Administrator** and add the Tailscale interface rule to the firewall:

```
New-NetFirewallRule -DisplayName "Rescue Backend Tailscale" -Direction Inbound -LocalPort 3001 -InterfaceAlias "
```

- Run CMD, navigate to the workspace, and run the synchronizer:

```
python sync_apps.py
```

- Start the server (`npm start`). Your phone can now submit SOS reports and receive live map updates from miles away over standard cellular networks!

---

3. Error Fixing Guide

This section is a troubleshooting handbook designed to help students, developers, and administrators identify, understand, and practically resolve common deployment and coding errors.

Troubleshooting Index:

3.1. Issue #1: Mismatch in Java Compiler Versions (The "Java 26" Crash)

- **Category:** Compilation / Environment Error.

- **Symptom:** During Gradle APK compilation, the terminal throws an immediate parse error: `java.lang.IllegalArgumentException: 26.0.1` inside `org.jetbrains.kotlin.com.intellij.util.lang.JavaVersion.parse`.

- **Root Cause:**

- Your Windows computer is running Java JDK 26 as its default system compiler.

- The React Native and Expo build plugins utilize older Kotlin compiler packages.

- These Kotlin version-parsing tools were coded before newer Java releases and cannot parse double-digit version strings higher than `21`. Seeing `"26.0.1"` triggers a parser crash, halting the build.

- **Step-by-Step Rectification:**

We must force the local terminal session to compile using **JDK 17** (standard stable version) without altering or uninstalling your global JDK 26.

1. Open PowerShell: Press the **Windows Key**, type **`powershell`**, and press **Enter**.

2. Locate Adoptium JDK 17: Check if the JDK 17 folder exists at: `C:\Program Files\Eclipse Adoptium\jdk-17.0.19.10-hotspot`

3. Navigate to the Project Folder: Run the command:

```
cd "C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026\rescuer-app\android"
```

4. Override JAVA_HOME and Compile: Define the variable specifically for this session and start compile:

```
$env:JAVA_HOME="C:\Program Files\Eclipse Adoptium\jdk-17.0.19.10-hotspot"
.\gradlew.bat assembleRelease
```

- **Verification:** The build console will show a green `BUILD SUCCESSFUL` message after packaging the release binary.

3.2. Issue #2: Windows Defender Port Blockage (The "Infinite Spinner" Network Block)

- **Category:** Network / Security Blockage.

- **Symptom:** The phone app is installed, but triggering a distress SOS displays an infinite loading spinner or throws a `Network Request Failed` error.

- **Root Cause:**

- Windows Defender Firewall secures your server from untrusted networks.

- By default, Windows blocks all unsolicited incoming network connections to safeguard your system.

- The Express server listens on port `3001`. Since your mobile phone is connecting over Wi-Fi, the firewall intercepts and silently drops the incoming packets on port 3001.

- **Step-by-Step Rectification:**

We must add a custom "Inbound Rule" that explicitly instructs the firewall to permit local TCP network traffic on port 3001.

1. Open PowerShell as Administrator:

- Press **Windows Key**, type **`powershell`**.

- **Right-click** *Windows PowerShell* and select **`Run as Administrator`**.
- Click **Yes** on the pop-up warning window.

2. Execute Inbound Rule Command: Paste the following command and press **Enter**:

```
New-NetFirewallRule -DisplayName "Rescue System Backend (Port 3001)" -Direction Inbound -LocalPort 3001 -Protocc
```

3. Anatomy of the Command Parameters:

- `-DisplayName "..."`: The name of the rule in the firewall dashboard.
- `-Direction Inbound`: Intercepts traffic coming *into* the computer.
- `-LocalPort 3001`: Specifies port 3001 where the Express server listens.
- `-Protocol TCP`: Restricts the rule to transmission control protocol packets.
- `-Action Allow`: Instructs the firewall to permit the connection.

4. Verification: Double-click the preconfigured **`Fix_Firewall.bat`** file in the root workspace folder to apply this rule automatically.

3.3. Issue #3: Router IP Address Reallocation (The "Lost Server Connection" Issue)

- **Category:** DHCP Network Configuration.
- **Symptom:** Everything worked perfectly yesterday, but today the mobile app returns a `Connection Timed Out` error when launched, even though the server is running.
- **Root Cause:**
- Your Wi-Fi router uses **DHCP (Dynamic Host Configuration Protocol)** to dynamically assign local IP addresses.
- When your computer disconnects or sleeps, the router may change your PC's IP address (e.g. from `192.168.1.4` to `192.168.1.8`).
- The phone is still trying to send requests to your old IP address (`192.168.1.4`), leading to timed-out connection requests.
- **Step-by-Step Rectification:**

1. Open Command Prompt (CMD): Press the **Windows Key**, type **`cmd`**, and press **Enter**.

2. Run IPCONFIG: Type `ipconfig` and note the new `IPv4 Address` on your wireless adapter.

3. Run Dynamic Synchronizer: Navigate to the root workspace and run:

```
cd "C:\Users\Alienware\Desktop\Rescue Backup 26-04-2026"
python sync_apps.py
```

- **Verification:** The script automatically re-scans the adapters, updates the static API routes inside all mobile code files, and recompiles the assets.

3.4. Issue #4: Low-Network Variable Scoping Mismatch (The "Offline Submission" Crash)

- **Category:** JavaScript Scoping / Runtime Error.
- **Symptom:** When a user disables Wi-Fi to test offline submission and taps the "SOS" button, the application crashes and shuts down immediately instead of queueing the request.
- **Root Cause:**
- Inside the application code, the `payload` variable was declared inside the `try` block:

```
try {
let payload = { category: 'fire', gps: coord }; // Scoped inside try
await sendRequestToServer(payload);
} catch (error) {
console.log("Failed to submit request: ", payload); // <-- CRASH!
}
```

- In JavaScript, variables declared with `let` are block-scoped. They only exist inside the curly braces `{}` they were created in.

- When the network goes offline, the `sendRequestToServer` function throws an error, causing the execution to jump directly to the `catch` block.

- Because `payload` does not exist outside the `try` block, the catch block throws a `ReferenceError` ("payload is not defined") and crashes the entire app.

- **Step-by-Step Rectification:**

We must lift (hoist) the variable declaration out and before the `try` block so it is visible to both the `try` and `catch` scopes.

1. Locate target file: Open `public-sos-app/App.js` in a text editor.

2. Alter Variable Scope: Declare `payload` outside and initialize as `null` first:

```
let payload = null; // Scoped outside try/catch
try {
payload = { category: 'fire', gps: coord };
await sendRequestToServer(payload);
} catch (error) {
console.log("Failed to submit request: ", payload); // <-- SAFE!
await queueOfflineSOS(payload); // Queues safely without crashes
}
```

- **Verification:** Run offline test; the app will safely queue the SOS and show a friendly offline alert banner without crashing.

3.5. Issue #5: Strict Database Filtering of Historical Data (The "Blank History" Issue)

- **Category:** Database SQL Query / API Logic.

- **Symptom:** The Rescuer App "History Page" remains completely blank or fails to load previously completed or declined rescue assignments.

- **Root Cause:**

- The Express.js backend had a strict SQL filter designed to hide finished tasks from active patrol tracking:

`SELECT * FROM rescue_requests WHERE status NOT IN ('completed', 'resolved', 'declined')`

- While this kept the web dashboard map clear of old tasks, it also blocked the history endpoint from accessing historical data.

- **Step-by-Step Rectification:**

We updated the API routing layer to differentiate active map queries from user-specific history queries.

1. Identify the Backend Router: File [server.js](file:///c:/Users/Alienware/Desktop/Rescue%20Backup%2026-04-2026/system-backend/server.js).

2. Modify database query: Implement a dedicated history endpoint:

```
// Endpoint retrieves all user history, including resolved entries
app.get('/api/users/:id/combined-history', (req, res) => {
const userId = req.params.id;
db.all("SELECT * FROM rescue_requests WHERE rescuer_id = ? ORDER BY timestamp DESC", [userId], (err, rows) => {
if (err) return res.status(500).json({ error: err.message });
res.json(rows);
});
});
```

- **Verification:** Completed and declined assignments are now rendered on the rescuer mobile history screen.

3.6. Issue #6: Destructive Cache Clearing Routines (The "Auto-Logout" Incident)

- **Category:** Data Storage Management.

- **Symptom:** Tapping the "Clear Cache" button on the phone's settings page correctly clears the offline history list, but also instantly logs the rescuer out of the app.

- **Root Cause:**

- The developer used `AsyncStorage.clear()` or `localStorage.clear()` inside the settings panel.

- This function wipes **every single key** stored by the application in the device database.

- Because the user's active login token and profile details were stored in the same local database, they were deleted along with the cache, triggering an unintended logout.

- **Step-by-Step Rectification:**

We must target specific keys during data clear operations instead of running a destructive global sweep.

1. Locate storage functions: Inside mobile preview files.

2. Implement Selective Clearing: Use `removeItem` specifically on data logs, keeping credentials intact:

```
// Safe Cache Clearing Routine
const clearAppCache = async () => {
// Target only data keys
await AsyncStorage.removeItem('cached_my_history');
await AsyncStorage.removeItem('offline_sos_queue');

// DO NOT delete 'citizen_user' or 'citizen_token'
console.log("Cache cleared safely. User remains logged in.");
};
```

- **Verification:** Triggering cache clear empties the history grid while keeping the active rescue profile session online.

3.7. Issue #7: Private NIB IMSI Authentication Failure or Radio Mismatch (The "No Service" Cellular Block)

- **Category:** Cellular Core / SIM Configuration Mismatch.

- **Symptom:** The mobile phone equipped with the private NIB-SIM card shows "No Service" or "Emergency Calls Only", and fails to attach to the private cellular base station.

- **Root Cause:**

- **IMSI / Key Mismatch:** The IMSI, secret key (`K`), or Operator Key (`OPc`) programmed into the physical USIM card does not exactly match the database records inside the LTE/5G Mobile Core (`open5gs-db` or `user_db.csv` in srsRAN).
- **PLMN ID Mismatch:** The SIM's MCC/MNC (e.g. `001/01`) differs from the PLMN broadcasted by the Software Defined Radio.
- **Radio Frequency Band Mismatch:** The SDR is configured to broadcast on an LTE band (e.g., Band 20 or Band 40) that the physical antenna or the Android smartphone's internal modem does not support.
- **Step-by-Step Rectification:**

We must align the card writing profile with the mobile core database and configure the transceiver to broadcast on a globally compatible radio channel.

1. Locate and Verify SDR Supported Bands:

- Open your srsRAN or Open5GS eNodeB configuration file (`enb.conf` or similar) in WSL2/Ubuntu.
- Verify the DL/UL earfcn frequencies match a band supported by your handset (e.g., Band 3 - 1800MHz or Band 7 - 2600MHz are highly compatible with commercial global smartphones).

2. Validate SIM Profiles: Connect the USB Smart Card Reader to your computer, open the SIM writing software, and read the programmed keys. Verify that MCC = `001`, MNC = `01` (or your private PLMN).

3. Insert Subscriber into NIB Core Database:

- *For Open5GS:* Open the WebUI in a browser at `http://localhost:3000` (or the WSL2 IP). Navigate to **Subscribers** -> **Add New Subscriber**. Input the exact IMSI (e.g. `001010000000001`), security keys `K`, `OPc`, and set APN profile to `rescue.nib`. Click **Save**.
- *For srsRAN:* Edit `/etc/srsran/user_db.csv` and append a line with the IMSI, authentication mode (milton/milenage), Ki, and OPc values.

4. Force Carrier Search on Mobile: On the phone, go to **Settings** -> **Network & Internet** -> **Mobile Network** -> Disable **Automatically Select Network**. Let it scan for 1-2 minutes, select the private network name manually, and confirm registration.

- **Verification:** The phone's network bar will change from "No Service" to showing active "LTE" or "5G" with full signal strength.

3.8. Issue #8: TUN Interface Isolation or Subnet Routing Blockage (The "Connected, No API Telemetry" Issue)

- **Category:** Operating System Core Routing / Firewall Isolation.
- **Symptom:** The Android phone successfully registers to the private cellular carrier (shows LTE/5G icon with strong signal) and obtains an IP address (e.g., `10.45.0.2`), but the ARDMS app cannot transmit reports to the server, and the web admin doesn't see the rescuer.
- **Root Cause:**
- **WSL2 Network Isolation:** The open-source cellular core runs inside a Linux virtual machine (WSL2), which acts as a NAT network behind Windows. The cellular packets reach the WSL2 interface but cannot traverse or bridge into the Windows host backend on port 3001.
- **Host IP Bind Error:** The Express.js backend was booted without specifying the TUN interface's local address, meaning it is only listening on `127.0.0.1` (localhost) or the primary LAN Wi-Fi IP instead of the `Open5GS-TUN` interface gateway (`10.45.0.1`).

- **Windows Defender Firewall Rule Mismatch:** The firewall is configured for a Wi-Fi connection but blocks inbound packets originating from the virtual TUN interface.

- **Step-by-Step Rectification:**

We must bind the backend server specifically to the TUN interface gateway and configure the host firewall to allow routing between the cellular subnet and the Express server.

1. Verify Backend Binding on Server:

- Ensure the Express backend binds to all active interfaces (`0.0.0.0`) or explicitly the TUN gateway (`10.45.0.1`).

- Run `python sync_apps.py 10.45.0.1` inside your Windows command terminal. This updates the mobile app API endpoints to target the static core gateway.

2. Enable Packet Forwarding in WSL2: If running the EPC/5GC core in WSL2, open the WSL2 terminal and run:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

3. Inspect/Create Windows TUN Firewall Rule:

- Open **PowerShell as Administrator** and add a firewall exception specifically for the private NIB IP subnet range:

```
New-NetFirewallRule -DisplayName "Rescue NIB Core Subnet" -Direction Inbound -LocalPort 3001 -Protocol TCP -Remo
```

4. Diagnose with PowerShell Interface Checks: Run `Get-NetIPInterface` to confirm the status of the `Open5GS-TUN` interface is active and routing.

- **Verification:** Running `ping 10.45.0.1` in the Termux app on the phone will return clean replies, and submitting an SOS incident will immediately record the event in the host database.